VERIFIED CODEBASE GUIDE & REFERENCE MANUAL

Understanding My Website

File-by-File Documentation & AI Token-Saving Handbook

Project Name: Tempo Event Website Builder
Core Architecture: React 18, Vite 6, TypeScript 5 & Tailwind CSS 3
Backend Architecture: Embedded SQLite (better-sqlite3) & Supabase Client
Target Audience: Beginners learning modern web development with AI
Workspace Path: `C:\Users\Lenovo\Desktop\Other\d`
Status: 100% verified against actual project files

Table of Contents

1. **Part 1: Understand My Entire Project**
   - 1.1 Verified Technology Stack
   - 1.2 Complete Project Folder Hierarchy Tree
2. **Part 2: Deep Dive File-by-File Explanation**
   - 2.1 Root Configuration & Tooling Files
   - 2.2 Backend & API Architecture (`server/` , `api/` , `data/`)
   - 2.3 Core Application Files (`src/main.tsx` , `src/App.tsx`)
   - 2.4 Application Pages & Views (`src/pages/`)
   - 2.5 UI Component Hierarchy (`src/components/`)
   - 2.6 Event Templates Engine (`src/templates/`)
   - 2.7 State Management, Services & Utilities (`src/context/` , `src/services/` , `src/utils/`)
   - 2.8 Static Assets & Public Files (`public/`)
3. **Part 3: Identify Unnecessary Files Safely**
   - 3.1 Safe Cleanup Assessment Table
   - 3.2 Step-by-Step Safe Cleanup Plan
4. **Part 4: Explain How My Website Runs**
   - 4.1 Runtime Flowchart: From Dev Command to Screen Pixels
   - 4.2 Exact Terminal Commands for Everyday Use
5. **Part 5: AI Token-Saving Guide**
   - 5.1 Token Mechanics: Input, Output, and Context Windows
   - 5.2 The 10 Golden Rules to Save 80% AI Tokens
6. **Part 6: Ready-to-Use AI Prompts (Copy & Paste)**
7. **Part 7: VS Code Beginner Workflow & Windows Shortcuts**
8. **Part 8: Practical Codebase Walkthroughs (3 Real Files)**
9. **Part 9: One-Page Quick-Reference Checklist & Beginner Glossary**

Part 1: Understand My Entire Project

1.1 Verified Technology Stack

Every technology below was directly verified by examining your project files (`package.json` , `vite.config.ts` , `tailwind.config.js` , and `server/`):

Layer / Category	Technology Detected	Role in Your Website
Frontend Framework	`React 18.3.1`	Builds your user interface using reusable component bricks (buttons, cards, forms, live preview frame).
Language & Types	`TypeScript 5.6.3` & `ES Modules`	Adds strict type checks to catch typos, missing variables, and bugs before code runs in the browser.
Build Tool & Bundler	`Vite 6.0.1`	Ultra-fast development server with instant Hot Module Replacement (HMR) and optimized production builder.
Styling & Design System	`Tailwind CSS 3.4.16` + `PostCSS`	Utility-first CSS styling directly in markup; includes custom color palettes (brand, coral) and glassmorphism.
Client-Side Routing	`react-router-dom 6.28.0`	Enables page switching (Home, Templates, Dashboard, Creator) smoothly without reloading the entire page.
Local Backend / API	`better-sqlite3` + Vite Dev Middleware	Runs an embedded API inside Vite's dev server; stores user accounts, templates, orders, and websites locally in `data/tempo.db` .
Cloud Backend (Optional)	`@supabase/supabase-js 2.117.2`	Optional hosted database and file storage for production cloud deployments.
Icons & Helpers	`lucide-react` , `qrcode` , `clsx` , `tailwind-merge`	Modern SVG icons, QR code generation for mobile event RSVPs, and safe Tailwind class merging.

1.2 Complete Project Folder Hierarchy

```
C:UsersLenovoDesktopOtherd
├── .env / .env.example       <-- Environment configuration (API keys, ports)
├── package.json              <-- Dependencies list and npm scripts
├── vite.config.ts            <-- Vite bundler config + custom API router middleware
├── tailwind.config.js        <-- Custom design tokens, colors, animations
├── tsconfig.json             <-- TypeScript compilation settings
├── index.html                <-- Single Page Application HTML root shell
│
├── api/                      <-- Serverless endpoint handlers (Vercel style)
│   ├── cron/payments.js
│   └── payments/             <-- Orders, approvals, and access controllers
│
├── server/                   <-- Local SQLite API server
│   ├── apiRouter.js          <-- Routes incoming /api requests
│   ├── db.js                 <-- SQLite schema and query helpers
│   ├── paymentRoutes.js      <-- Payment processing endpoints
│   └── payments.js           <-- Payment state management
│
```

```
├── data/
│   └── tempo.db              <-- Embedded SQLite database holding websites & users
│
├── public/                   <-- Static files served directly (images, audio, templates)
│   ├── html-templates/       <-- Standalone HTML/CSS event invitation pages
│   ├── music/                <-- Audio tracks for invitations
│   ├── robots.txt & sitemap  <-- SEO discovery files
│   └── upi-payment.gif       <-- Payment instructions visual guide
│
├── src/                      <-- Core React & TypeScript application source
│   ├── main.tsx              <-- App entry point; mounts React to DOM
│   ├── App.tsx               <-- Root router mapping all URL paths to pages
│   ├── index.css             <-- Global CSS base styles and Tailwind directives
│   ├── components/           <-- Modular UI blocks
│   │   ├── common/           <-- Buttons, Modals, Navbar, Footer, Toast
│   │   ├── dashboard/        <-- Website cards and stats displays
│   │   ├── editor/           <-- Tabbed website customize forms & live preview
│   │   └── templates/        <-- Template cards, filters, and preview modals
│   ├── context/              <-- Global state (AuthContext.tsx)
│   ├── pages/                <-- Full-screen route views (Home, Creator, Dashboard)
│   ├── services/             <-- API clients, Supabase, Local Storage, Events
│   ├── templates/            <-- Built-in interactive template engines & registry
│   ├── types/                <-- TypeScript interface definitions
│   └── utils/                <-- Helper functions (date formatting, RSVP links)
│
├── dist/                     <-- Built production output (generated by npm run build)
├── node_modules/             <-- Downloaded third-party packages (managed by npm)
└── [Loose Artifact Folders]  <-- Birthady party/, tempo/, stitch_*/ (Safe to clean)
```

Part 2: Deep Dive File-by-File Explanation

Below is the complete analysis of every file and directory verified in your actual project.

2.1 Root Configuration & Tooling Files

1. `package.json`

- **Path:** `/package.json`
- **Simple Explanation:** The ID card and blueprint of your project. It lists every external library your website uses, project metadata, and shortcut commands (like `npm run dev`).
- **Why it exists:** Node.js and Vite need to know which libraries to download and how to launch your project.
- **Which part uses it:** The entire development workflow, npm, and build tools.
- **Status:** Essential.
- **What breaks if modified:** Deleting it destroys your project's ability to install packages or run commands. Incorrect edits can cause dependency conflicts.
- **Beginner safety:** Do not touch manually unless adding a package via `npm install`.

2. `package-lock.json`

- **Path:** `/package-lock.json`
- **Simple Explanation:** A detailed security seal that records the exact version of every single file downloaded into `node_modules`.
- **Why it exists:** Ensures that when you or someone else runs `npm install` on another machine, identical code is installed down to the last digit.
- **Which part uses it:** `npm` during package installation.
- **Status:** Generated automatically.
- **What breaks if modified:** Inconsistent builds or unexpected version updates across installations.
- **Beginner safety:** Never edit this file by hand. Leave it alone.

3. `vite.config.ts`

- **Path:** `/vite.config.ts`
- **Simple Explanation:** The control center for Vite. It activates the React plugin, sets the local port to `3000`, creates the `@/` alias pointing to `src/`, and injects the `tempoBackendPlugin()` middleware so requests to `/api` connect directly to your local database without running a separate server process.
- **Why it exists:** Configures bundling rules and local server behavior.
- **Which part uses it:** Vite dev server and build pipeline.
- **Status:** Essential.
- **What breaks if modified:** The local website will stop loading, paths starting with `@/` will fail, and API calls to `/api` will return 404 errors.
- **Beginner safety:** Do not change unless specifically instructed to adjust port or proxy settings.

4. `tailwind.config.js` & `postcss.config.js`

- **Path:** `/tailwind.config.js`, `/postcss.config.js`
- **Simple Explanation:** Custom styling configuration. Contains custom color palettes (e.g. `coral`, `brand`), animations, and font settings. PostCSS ensures vendor prefixes are added for cross-browser compatibility.
- **Why it exists:** Allows you to write utility classes like `bg-brand-500` and compiles them into pure CSS.
- **Which part uses it:** Every component and page styled with Tailwind.
- **Status:** Essential.
- **What breaks if modified:** All custom colors, rounded corners, and animations across your website will revert to default or look broken.
- **Beginner safety:** Safe to leave alone. Only edit when you want to add new theme colors.

5. `tsconfig.json` & `tsconfig.node.json`

- **Path:** `/tsconfig.json`, `/tsconfig.node.json`
- **Simple Explanation:** Rules for TypeScript compiler. Tells TypeScript how strictly to verify your code and how to resolve imports like `@/components/...`.
- **Why it exists:** Enforces code health and type safety across your React components.
- **Which part uses it:** VS Code editor and the `tsc` build step.
- **Status:** Essential.
- **What breaks if modified:** VS Code will show red underline errors everywhere, and `npm run build` will fail.
- **Beginner safety:** Leave completely alone.

6. `index.html`

- **Path:** `/index.html`
- **Simple Explanation:** The master web page frame. Contains the `<div id="root"></div>` where React mounts the entire application, loads Google Fonts (Inter, Playfair Display), and has the script tag loading `/src/main.tsx`.
- **Why it exists:** Browsers only load HTML files directly. Vite uses this as its entry gate.
- **Which part uses it:** The browser entry point.
- **Status:** Essential.
- **What breaks if modified:** Deleting the `#root` div causes a completely blank white screen.
- **Beginner safety:** Safe to leave alone, or edit only to update the `<title>` tag or favicon.

7. `.env` & `.env.example`

- **Path:** `/.env`, `/.env.example`
- **Simple Explanation:** Secret key configuration. Contains sensitive credentials such as your Supabase Project URL and Anonymous Public Key. `.env.example` is a blank template showing what variables are required.
- **Why it exists:** Prevents hardcoding secret keys inside code files.
- **Which part uses it:** `src/services/supabase.ts` via `import.meta.env`.
- **Status:** Essential.
- **What breaks if modified:** Supabase authentication and database cloud storage will fail to connect.
- **Beginner safety:** Keep your `.env` file private. Never share it publicly or upload it to GitHub.

8. `firebase.json` & `.firebaserc` / `vercel.json`

- **Path:** `/firebase.json`, `/.firebaserc`, `/vercel.json`
- **Simple Explanation:** Deployment routing configs. They tell Firebase Hosting or Vercel to route all web traffic to `index.html` so React Router can handle pages on reload.
- **Why it exists:** Prevents 404 Not Found errors when refreshing sub-pages on production hosts.
- **Which part uses it:** Cloud hosting providers when deployed.
- **Status:** Used by the project (for deployment).
- **What breaks if modified:** Direct URL visits to `/dashboard` on deployed hosts would fail.
- **Beginner safety:** Leave alone.

2.2 Server & Backend Directory (`server/` & `data/`)

1. `server/apiRouter.js`

- **Path:** `/server/apiRouter.js`
- **Simple Explanation:** The API dispatcher. It intercepts any web request starting with `/api/` (such as `/api/auth/register`, `/api/events`, `/api/orders`) and calls the matching database logic in `server/db.js`.
- **Why it exists:** Provides a self-contained local backend without requiring you to run a separate Node server in a second terminal.
- **Which part uses it:** `vite.config.ts` middleware.
- **Status:** Essential for local database mode.
- **What breaks if modified:** Account creation, event saving, and payments will fail.

2. `server/db.js`

- **Path:** `/server/db.js`
- **Simple Explanation:** The SQLite database manager. Creates tables for `users`, `events`, `orders`, and `guestbook`, and provides helper functions to insert and query records.
- **Why it exists:** Direct interface to the local SQLite database file.
- **Which part uses it:** `server/apiRouter.js`.
- **Status:** Essential.

3. `data/tempo.db`

- **Path:** `/data/tempo.db`
- **Simple Explanation:** Your actual local database file. Stores all your saved birthday events, website customizations, and registered user accounts on your hard drive.
- **Why it exists:** Keeps your data persistent across website restarts.
- **Status:** Essential (Data Store).
- **Beginner safety:** Never edit this file with VS Code text editor; it is a binary database file.

2.3 Application Core (`src/`)

1. `src/main.tsx`

- **Path:** `/src/main.tsx`
- **Simple Explanation:** The ignite button. It finds the `#root` element in `index.html` and renders your `<App />` component inside React's StrictMode. Also imports `index.css`.
- **Status:** Essential.

2. `src/App.tsx`

- **Path:** `/src/App.tsx`
- **Simple Explanation:** The site-wide map. Configures all URLs using React Router:
  - `/` → `HomePage`
  - `/templates` → `TemplatesPage`
  - `/create/:templateId` → `WebsiteCreatorPage`
  - `/dashboard` → `DashboardPage` (Protected)
  - `/signin` & `/signup` → Authentication Pages
  - `/e/:slug` → `PublicEventPage` (Live published websites)
- **Status:** Essential.

3. `src/context/AuthContext.tsx`

- **Path:** `/src/context/AuthContext.tsx`
- **Simple Explanation:** The global user memory. Remembers whether you are currently logged in, stores your user profile, and provides `signIn()`, `signUp()`, and `signOut()` functions to all components.
- **Status:** Essential.

2.4 Key Feature Pages (`src/pages/`)

Page File	Simple Description & Purpose	Status
`HomePage.tsx`	The marketing landing page. Features hero banners, template highlights, testimonials, feature badges, and a call-to-action button.	Essential
`TemplatesPage.tsx`	The template catalog. Lets users search, filter by category (Birthday, Wedding, Portfolio), and preview invitation themes.	Essential
`WebsiteCreatorPage.tsx`	The flagship visual website builder. Has a live two-panel workspace: controls on the left (text, music, photos, RSVP) and a real-time updating live preview phone frame on the right.	Essential
`DashboardPage.tsx`	User dashboard showing all websites you have created, edit shortcuts, shareable links, and QR codes.	Essential
`PublicEventPage.tsx`	The published view that guests see when opening an event link (e.g. `mysite.com/e/sarah-10th-birthday`).	Essential
`SignInPage.tsx` & `SignUpPage.tsx`	Login and registration forms with validation and error toasts.	Essential
`AdminPaymentsPage.tsx`	Back-office interface to review UPI transaction proofs and approve premium template purchases.	Used by the project

2.5 Template Engine Architecture (`src/templates/`)

Your website builder contains rich, ready-to-use celebration themes:

- **`registry.ts`:** The central catalog defining each template's ID, title, category, thumbnail, default colors, and which React component renders it.
- **`NovaBirthdayWorldTemplate.tsx`:** High-energy, colorful celebration template featuring interactive confetti and guestbook.
- **`KeepsakeBirthdayTemplate.tsx`:** Elegant scrapbook-style birthday memory template with timeline and photo galleries.
- **`KomorebiDiningTemplate.tsx`** & **`SolsticeSageTemplate.tsx`:** Modern restaurant menu and botanical wedding celebration templates.
- **`HtmlIframeTemplate.tsx`:** Powerful fallback renderer that displays standalone HTML templates safely inside an isolated iframe.

Part 3: Identify Unnecessary Files Safely

During the codebase inspection, several non-essential, backup, and duplicate files were detected. Below is the safe assessment audit table.

File / Folder	Purpose	Status	Safe Action	Reason
`tempo/` (folder)	Old duplicate backup snapshot of the entire project.	**Possibly unused**	Archive to external backup folder or delete	It contains 110 redundant files duplicating your main root files.
`Birthady party/` (folder)	HTML prototyping artifacts and exported designs.	**Possibly unused**	Move to safe `_archive/`	Not imported by any file in `src/`. It contains experimental mockups.
`stitch_atelier_templates_marketplace/`	Standalone downloaded template kit.	**Possibly unused**	Move to safe `_archive/`	Templates have already been integrated into `src/templates/`.
`new index htmls/` (folder)	Draft HTML designs and poster videos.	**Possibly unused**	Archive or delete	Loose working drafts from earlier design iterations.
`test.html`	Temporary 20-byte scratch file.	**Possibly unused**	Delete safely	Contains no production code or references.
`share_image4113068545197211153 - Copy.gif`	Duplicate GIF in the root directory.	**Possibly unused**	Delete safely	Identical asset already exists at `public/upi-payment.gif`.
`dist/` (folder)	Compiled production build output.	**Generated automatically**	Leave alone (or clean with gitignore)	Automatically recreated every time you run `npm run build`.
`node_modules/` (folder)	Downloaded npm dependencies (11,500+ files).	**Generated automatically**	Never edit or commit	Can be re-downloaded at any time by running `npm install`.

Safe 3-Step Cleanup Plan (Never Delete Blindly!)

1. **Create a Backup:** Before moving or deleting any file, make a compressed ZIP backup of your entire folder, or create a git commit:

```
git add .
git commit -m "Backup before cleaning unused folders"
```

2. **Move to an `_archive/` Folder:** Instead of deleting, move `Birthady party/`, `tempo/`, and `new index htmls/` into a separate folder outside your project.
3. **Verify Local Build:** Run `npm run build` in your terminal. If the build succeeds with zero errors and your dev server runs smoothly, the files were 100% unused.

Part 4: Explain How My Website Runs

Here is what happens under the hood from the second you run the start command until pixels appear on your monitor.

4.1 The Lifecycle Flowchart



4.2 Exact Terminal Commands for Your Project

Task	Terminal Command	What It Does
Install Dependencies	`npm install`	Reads `package.json` and installs all required packages into `node_modules`.
Start Local Website	`npm run dev`	Launches Vite on `http://localhost:3000` with live auto-

		refresh as you edit files.
Check Code & Build	`npm run build`	Runs TypeScript type-checks (`tsc`) and compiles production assets into `dist/` .
Preview Production Build	`npm run preview`	Simulates your production deployment locally on your machine.

Part 5: AI Token-Saving Guide

5.1 Understanding Tokens: The Fuel of AI

AI models do not read files as traditional text or words; they read chunks of characters called **Tokens** (roughly 1 token ≈ 4 characters or 0.75 words in English). Everything exchanged with the AI has a token cost:

- **Input Tokens:** Every single word in your prompt PLUS every file or snippet you attach. *Pasting your whole project consumes massive input tokens on every turn.*
- **Context Window:** The AI's working memory limit. As your conversation gets longer, older messages and large pasted files accumulate, causing the AI to become sluggish, lose focus, or hallucinate.
- **Output Tokens:** The code and text the AI writes back to you. Generating full 500-line files consumes heavy output tokens and takes much longer to complete.

5.2 The 10 Golden Rules to Save 80% of AI Tokens

Rule 1: Never Paste Entire Files When Changing Small Sections

If you want to modify a button in `WebsiteCreatorPage.tsx` (which is 1,200+ lines long), do not paste all 1,200 lines. Paste only the button JSX and its handler function (approx. 20 lines). This saves over 95% of your tokens.

Rule 2: Ask for Diffs or Targeted Snippets, Not Full File Rewrites

Tell the AI: "*Show me only the modified function or a standard Git diff. Do not rewrite the entire file.*" This drastically cuts output tokens.

Rule 3: Have AI Diagnose Root Causes Before Writing Code

Instead of saying "*My payments are broken, rewrite my payment code*", ask: "*Here is the error message: [paste error]. What are the top 2 possible root causes in my logic?*" Fix the diagnosis first, then write 5 lines of code instead of 300.

Rule 4: Use VS Code Search to Find the Exact Needle

Press `Ctrl + Shift + F` to search for the specific button text or class name. Find the exact 10 lines responsible, and provide only those lines to the AI.

Rule 5: Reuse Existing UI Components Instead of Creating New Ones

Your codebase already has `Button.tsx`, `Modal.tsx`, `Input.tsx`, and `Badge.tsx` in `src/components/common/`. Tell AI: "*Use my existing Button and Modal components from `@/components/common/` rather than creating new custom elements.*"

Rule 6: Maintain a Concise System Context Note

Instead of explaining your entire tech stack every time, use a two-line summary: "*React 18 + Vite + TypeScript + Tailwind CSS. Path alias '@' points to '/src'.*"

Rule 7: Request 'Explanation Only on Request'

Tell the AI: "*Be concise. Return only the updated snippet and a 1-sentence summary of why it solves the problem.*" This saves hundreds of conversational output tokens.

Rule 8: Inspect Imports Before Writing Duplicate Helper Utilities

Check `src/utils/`. You already have `cn.ts`, `formatting.ts`, and `countdown.ts`. Reusing them keeps your codebase clean and token spend low.

Rule 9: Start Fresh AI Conversations for Unrelated Tasks

Do not keep a single chat open for weeks. Once a feature is done, start a new chat so previous conversations do not count against your input token quota.

Rule 10: Use Terminal Commands to Filter Bug Information

Instead of pasting 500 lines of terminal logs, copy only the top 5 lines of the error stack trace showing the filename and line number.

Part 6: Ready-to-Use AI Prompts

Copy and paste these battle-tested, token-efficient prompt templates directly into Gemini or your AI assistant.

Prompt 1: Understanding an Unfamiliar File

```
"Explain the purpose of [FILENAME] in 3 bullet points: 1) What it does, 2) Which other files depend on it, and 3) Whether it is safe for a beginner to modify. Keep your answer under 100 words."
```

Why it saves tokens: Limits output length strictly and prevents long generic lectures.

Prompt 2: Finding Files Responsible for a Feature

```
"In my React+Vite project, I want to modify [FEATURE NAME, e.g. the RSVP WhatsApp share button]. Which file and function contains this logic? List only the file paths and line ranges."
```

Why it saves tokens: Produces an instant roadmap without generating unnecessary code.

Prompt 3: Fixing a Bug Without Rewriting Everything

```
"Here is the error message from my terminal: [PASTE ERROR]. Below is the relevant 15-line function from [FILENAME]: [PASTE FUNCTION]. What is the root cause? Return only the corrected lines to replace."
```

Why it saves tokens: Targets only the broken lines, avoiding entire file regenerations.

Prompt 4: Adding a Feature with Existing Components

```
"I want to add a [NEW FEATURE, e.g. confirmation popup] to [FILENAME]. Use my existing Modal and Button components from '@/components/common/'. Show only the import line and the new JSX snippet."
```

Why it saves tokens: Prevents AI from generating duplicate modal components or huge files.

Prompt 5: Finding Potentially Unused Files Safely

```
"I am auditing my project to remove clutter. Check if [FILENAME] is imported anywhere in 'src/' or referenced in configs. Answer with either 'ACTIVE' or 'CANDIDATE FOR CLEANUP' and a 1-sentence reason."
```

Prompt 6: Asking AI to Explain Proposed Changes First

```
"Before writing any code, outline your proposed plan in 3 steps to implement [CHANGE]. Do not write the code yet. Wait for my approval."
```

Why it saves tokens: Prevents generating large amounts of incorrect code that has to be discarded.

Part 7: VS Code Beginner Workflow

Master these core skills in VS Code to inspect your codebase effortlessly without relying on AI for simple lookups.

7.1 Visual Studio Code Key Shortcuts (Windows)

Shortcut	Action	Beginner Use Case
`Ctrl + P`	**Quick Open File**	Type any file name (e.g. `App.tsx`) to jump straight to it instantly.
`Ctrl + Shift + F`	**Global Search**	Search for any text, function name, or CSS class across your whole project.
`F12`	**Go to Definition**	Click on any component or function and press `F12` to jump to where it was created.
`Shift + F12`	**Find All References**	See every file in the project that uses this specific component.
`Ctrl + ` ` (Backtick)	**Toggle Terminal**	Opens or hides your terminal window to run `npm run dev`.
`Ctrl + /`	**Toggle Line Comment**	Quickly disable a line of code for testing without deleting it.
`Ctrl + Shift + G`	**Source Control (Git)**	See a colored side-by-side diff of every file changed by AI.

7.2 How to Safely Review AI Changes Before Accepting

1. **Open Source Control Panel (`Ctrl + Shift + G`):** Whenever files are changed, VS Code lists them under "Changes".
2. **Click the File Name:** VS Code opens a split-screen comparison:
   - **Red (Left side):** The original code that was removed.
   - **Green (Right side):** The new code added by the AI.
3. **Verify the Changes:** Check that your existing comments were preserved and no important functions were accidentally wiped out.
4. **Test in Browser:** Check your browser window at `http://localhost:3000` to verify everything looks and behaves as expected.

Part 8: Practical Examples Using My Actual Project

Here are three real-world walkthroughs examining active files from your project.

Walkthrough 1: `src/components/common/Button.tsx`

- **Understanding the File:** A reusable button component. Supports 4 visual styles (`primary` , `secondary` , `outline` , `ghost`) and 3 sizes (`sm` , `md` , `lg`). Uses the `cn()` helper to merge Tailwind classes cleanly.
- **Tracing Connections:** Imported by `Navbar.tsx` , `WebsiteCreatorPage.tsx` , `TemplateCard.tsx` , and `Modal.tsx` . It is the visual building block for every button on your site.
- **Token-Efficient AI Prompt:**

```
"In src/components/common/Button.tsx, I want to add a 'danger' variant with red styling. Show me only the new variant object to add to the variants dictionary."
```

- **How to Verify:** Run `npm run dev` , open any page, and pass `variant="danger"` to a button to confirm it displays red.

Walkthrough 2: `src/context/AuthContext.tsx`

- **Understanding the File:** The security and user state center. Manages user login tokens in `localStorage` , communicates with `/api/auth` or Supabase, and exposes `user` , `signIn` , and `signOut` .
- **Tracing Connections:** Wraps your entire app inside `App.tsx` . Used by `ProtectedRoute.tsx` to lock the dashboard from unauthorized visitors.
- **Token-Efficient AI Prompt:**

```
"In src/context/AuthContext.tsx, which function handles logout? Show me only the signOut function definition and how it clears localStorage."
```

- **How to Verify:** Click "Sign Out" in your navigation bar; verify that you are redirected to the login page and your token is cleared.

Walkthrough 3: `src/templates/registry.ts`

- **Understanding the File:** The master template catalog array. Holds definitions for every theme: ID, title, description, category, tags, and which component renders it.
- **Tracing Connections:** Consumed by `TemplatesPage.tsx` to render the catalog grid and by `WebsiteCreatorPage.tsx` to load the chosen theme.
- **Token-Efficient AI Prompt:**

```
"Here is the structure of one item from TEMPLATES in src/templates/registry.ts. Generate a new entry for a 'Retro 90s Disco' theme following the exact same schema. Return only the object snippet."
```

- **How to Verify:** Save the file and visit `http://localhost:3000/templates` . Your new theme card will automatically appear in the list!

Part 9: One-Page Quick Reference Checklist & Glossary

9.1 Daily Development Checklist

Step	Command or Action
1. Open Project	Open the folder `C:\Users\Lenovo\Desktop\Other\d` in VS Code.
2. Start Local Server	Press `Ctrl + ``, type `npm run dev`, press Enter.
3. Open in Browser	Navigate to `http://localhost:3000` in Chrome or Edge.
4. Before Asking AI	Locate the specific file with `Ctrl + P`. Copy only the relevant 10-20 lines.
5. Before Risky Changes	Commit changes or create a quick ZIP backup of your project folder.
6. Check for Errors	Run `npm run build` in the terminal to verify zero TypeScript errors.

9.2 Beginner Glossary of Terms

- **Component:** A self-contained, reusable piece of user interface (like a Navbar, Button, or Card) written in React.
- **Vite:** The ultra-fast development server and asset packager that runs your local website.
- **HMR (Hot Module Replacement):** Vite's feature that instantly updates changes in your browser without requiring a full page refresh.
- **Props:** Arguments passed into a React component (like passing a title or button color).
- **State:** Internal component memory that triggers a screen re-render whenever its value changes (e.g. text typed in a form).
- **Context:** Global memory shared across the entire website (like user login state) without having to pass props through every component level.
- **Tailwind CSS:** A styling system that lets you design directly inside HTML tags using utility classes like `text-lg font-bold p-4`.
- **SQLite (tempo.db):** A lightweight, file-based database that stores your website data locally on your computer without needing complex server setups.